

2.1: Record or Tasks

Task #	Date	Phase	Description	Duration
1	Nov 21	Planning	First interview to establish problem and gather requirements from client	10 minutes
2	Nov 29	Planning	Finalized success criteria according to the first interview with client.	30 minutes
3	Nov 30	Development	Set up git, github, Qt Creator, and core project structure for program	30 minutes
4	Dec 4	Development	Added sqlite3 as a library to the project. Ran into several errors due to existing library conflicts, linkage errors, and syntax	3 hours
5	Dec 7	Development	Finished database class constructor. Ran into some SQL syntax errors	1 hour
6	Dec 8	Design	Designed UI for the different windows in my application. Then implemented it into code without any actual back-end functionality	1 hour
7	Dec 8	Development	Began implementing the ability to add vocabulary lists. Ran into several issues with pointers and being able to hand over member variables	2 hours
8	Dec 9	Development	Finished implementing adding vocabulary lists. Ran into more issues with pointers.	1 hour
9	Dec 10	Development	Began adding the ability to add words to a list. Also began adding the supermemo algorithm.	1 hour

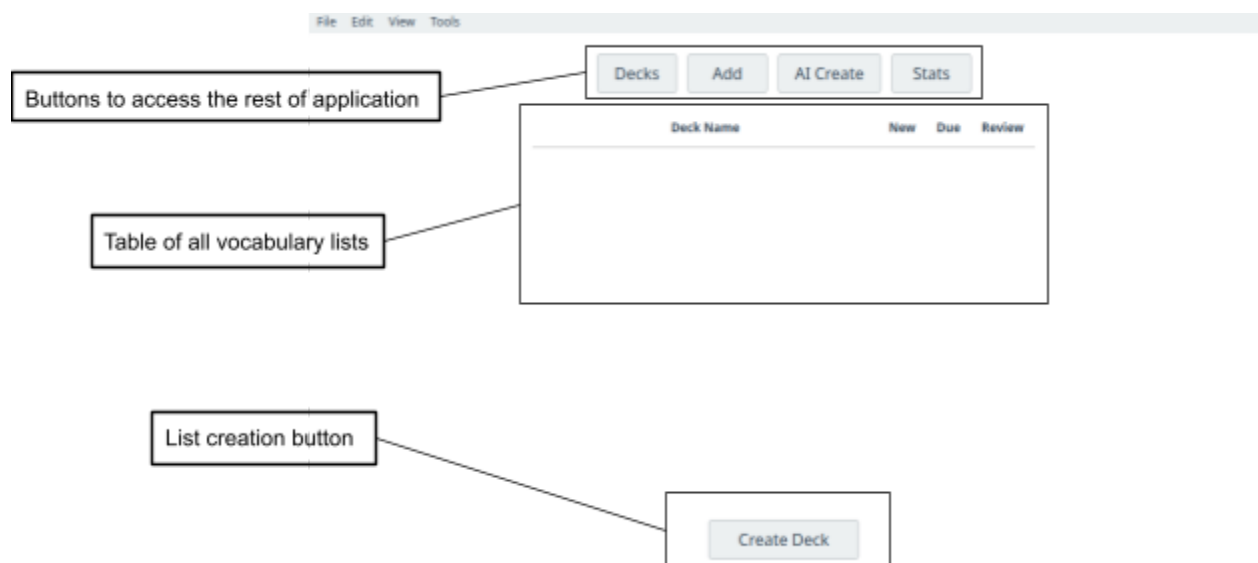
Task #	Date	Phase	Description	Duration
10	Dec 12	Design Planning	Added more structure to the project directory. Made sure everything uploaded to github	1 hour
11	Dec 14	Design	Designed the main windows of the application, trying to keep them simple from the beginning	1 hour
12	Dec 14	Design	Worked on flowcharts for application flow	1.5 hours
13	Dec 15	Development	Added functionality for studying words such as displaying lists, getting the words, etc. Ran into issue making a new window	3 hours
14	Dec 16	Development	Added more than one study method. Ran into issue when switching between multiple panels	2 hours
15	Dec 16	Development	Ensured the easiness factor was always in the accepted range	30 minutes
16	Dec 17	Development	Added the option to view all the words and definitions in a list	1 hour
17	Dec 17	Development	Began adding the ability to use ai for the creation of a list. Ran into issues with using environmental variables	3 hours
18	Dec 18	Development	Added two distinct themes for the user to be able to use. Ran into issues with style sheets because of syntax	2 hours
19	Dec 18	Development	Worked on making sure the user can't add multiple of the same word to the same list	1 hour

Task #	Date	Phase	Description	Duration
20	Dec 18	Development	Cleaned up the ui so it wouldn't look so jagged. Again had to look into how to use style sheets and different Qt methods to get everything to work smoothly	1 hour
21	Dec 19	Development	Tried to work more on AI features, such as API keys and which model to use	1 hour
22	Dec 26	Development	Added another study method for the user. Ran into issues displaying whether the user was correct.	2 hours
23	Dec 26	Development	Made sure everything was displayed correctly. Fixed bugs where the ui wasn't updating	1 hour
24	Dec 26	Development	Made sure bug/error handling is done through native Qt methods to make sure they work all the time	30 minutes
25	Dec 31	Design	Worked on UML diagram for all of the classes in my program	1 hour
26	Jan 1	Design	Added the ERD diagram of the sql database after generating it with Database Visualizer	30 minutes
27	Jan 24	Design	Started adding the testing plan for the success criteria	30 minutes
28	Jan 30	Development	Beginning to add the ability to import csv files to create a list. Ran into issues with incorrect parsing of file	30 minutes

Task #	Date	Phase	Description	Duration
29	Feb 04	Development	Finished implementing importing through csv files. Main issues were multiple lines being parsed	1 hour
30	Feb 05	Development	Worked on an edge where buttons would show before it was necessary.	30 minutes

2.2: GUI

Fig. 2.2.0: Starting page when application is opened



As per criteria nine, I decided to keep the elements to a minimum. The buttons at the top are used to access the rest of the applications features, such as adding a word to a list. The table will display the list name and how many words to study. The button at the bottom is kept to allow for easy list creation

Fig. 2.2.1: List creation window

The diagram illustrates a 'List creation window' with the following components and annotations:

- Entering name of list:** Points to the 'List Name:' text label and its corresponding single-line text input field.
- Entering target language for study:** Points to the 'Target Language:' text label and its corresponding single-line text input field.
- Description of list:** Points to the 'Description:' text label and its corresponding multi-line text area.
- Creation or cancellation of new list:** Points to the 'Create List' and 'Cancel' buttons located at the bottom right of the window.

The window itself is a rectangular container with a light gray border. It contains three stacked input fields (two single-line and one multi-line) and a button group at the bottom right.

Again, I went with a simple design. I decided to use text fields for every single field to make sure multiple words can be stored, and to make sure that the user can use whatever script as long as it can be recognized by the computer's default system to satisfy success criteria two.

Fig. 2.2.3: Word addition window

The diagram illustrates a 'Word addition window' with the following components and callouts:

- Choosing the list to add a word to:** Points to the 'List:' dropdown menu.
- The word to be added to the list:** Points to the 'Word:' text input field.
- Dropdown for additional options:** Points to the 'Definition:' text area.
- Additional options apart from just the definition:** Points to the 'Additional Options' section, which includes:
 - 'Example:' text input field
 - 'Notes:' text input field
 - 'Part of Speech:' dropdown menu
 - 'Select Option' button
- Confirmation or cancellation:** Points to the 'Add' and 'Cancel' buttons at the bottom right.

This window has the additional options hidden by default to keep it simple, and the user can choose to add additional options. The last box is for extra relations such as synonyms or antonyms to satisfy success criteria six.

Fig 2.2.4: Creation of a list through the use of AI

The diagram illustrates the 'AI Create Vocabulary List' window. It features a title bar at the top. Below the title, there are five input fields: a text box for the list name, a dropdown menu for the AI model (currently showing 'gpt-3.5-turbo'), a large text area for optional prompt instructions, a numeric spinner for the number of words (set to 10), and a confirmation/cancellation section at the bottom right containing 'Generate' and 'Cancel' buttons. Callout boxes on the left side of the window identify each of these components.

AI Create Vocabulary List

Name of new list

List name

Model of AI chosen

gpt-3.5-turbo

Extra details for list generation

Optional prompt instructions for the model

Number of words to be generated in the list

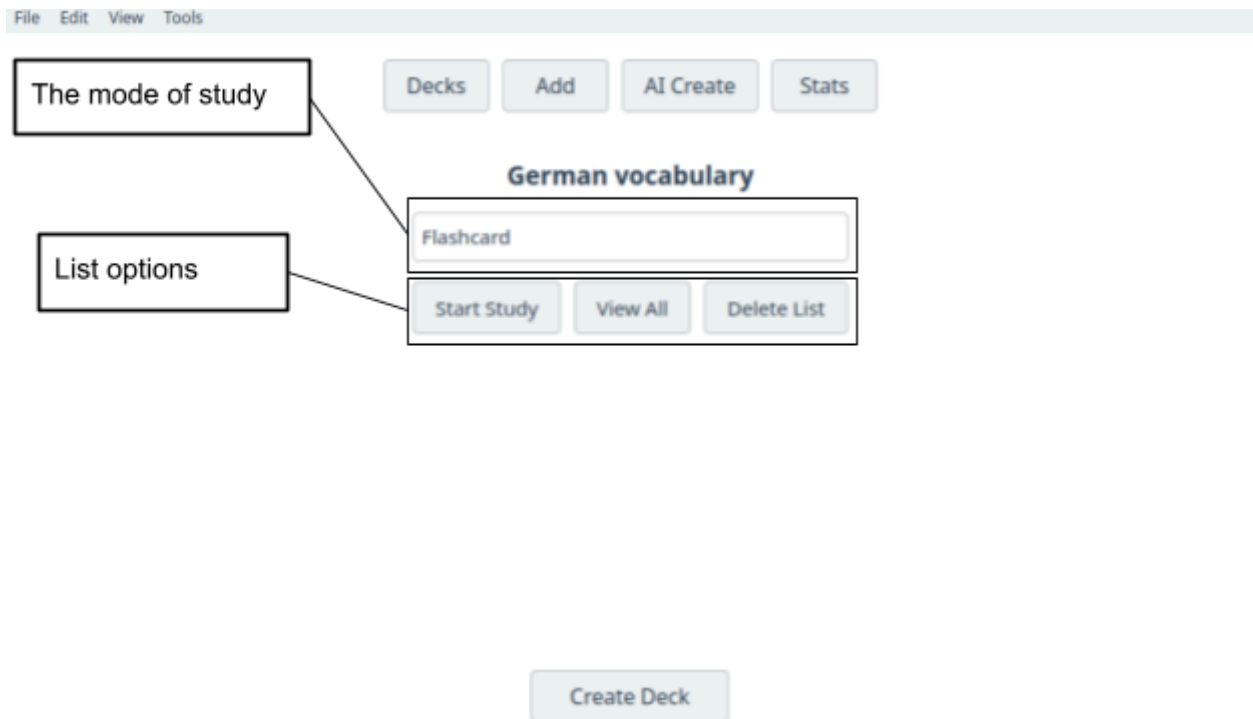
10

Confirmation or cancellation

Generate Cancel

This window allows for the user to choose the list name, the model of AI they would like to utilize, any additional instructions to the model, and the numbers of words to add to the list. It is similar to the regular list creation window, just with AI this time to help with success criteria nine.

Fig 2.2.5: Choosing the mode of study as well as deletion of a list in a second panel



This is the window that appears after a user double-clicks on a list from the table in the main window. It includes a dropdown for the three different study methods, and three buttons for list options. I decided to keep the buttons at the top to allow the user to go back to the main window without any extra motions.

Fig 2.2.6: Flashcard study method panel



This is the flashcard study method which has the word to be studied, a button revealing the definition, and four buttons that the user can use to rate how difficult the word was to recall. This information would later be used to calculate future times of study, supporting success criteria five.

Fig 2.2.7: Multiple Choice study method panel



Multiple choice study method. This window will utilize other word definitions within the list to make things difficult to choose. The additional information box will show all information available about the word other than the word itself.

Fig 2.2.8: Typing study method panel

The image shows a software interface for a typing study method. At the top is a menu bar with 'File', 'Edit', 'View', and 'Tools'. Below the menu bar are four buttons: 'Decks', 'Add', 'AI Create', and 'Stats'. The main content area is titled 'Hallo'. It contains a text input field with the placeholder text 'Type the definition:' and a 'Submit' button below it. Two callout boxes are present: one labeled 'Input of word definition' pointing to the text input field, and another labeled 'Submission button' pointing to the 'Submit' button.

This is the window for the typing study method. Similar to the previous, the only thing that changes is an input box for the answer to the word definition. The window will display information about the word if the user gets the definition wrong and they will make another attempt to get it correctly.

2.3: Flowcharts

Fig 2.3.0: General System

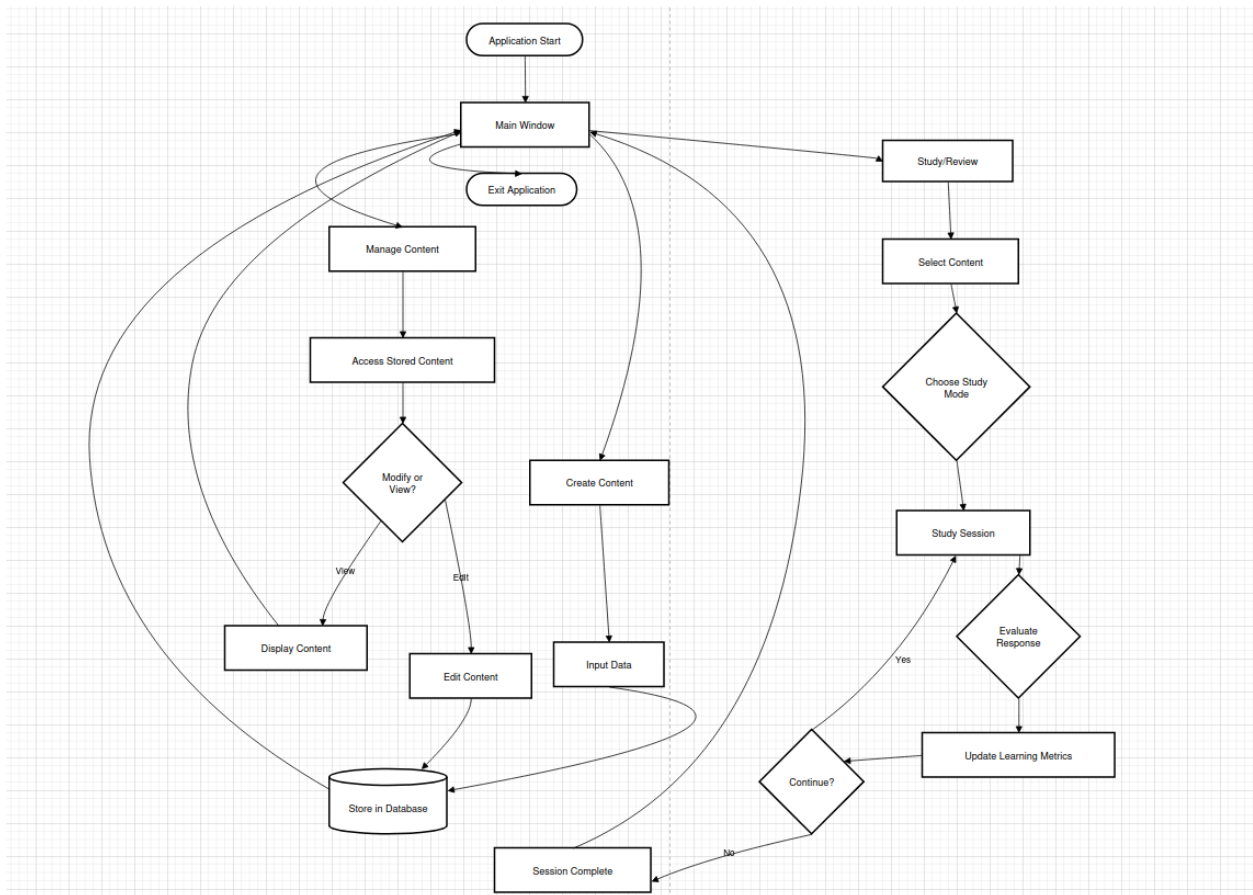


Fig 2.3.1: Studying a word

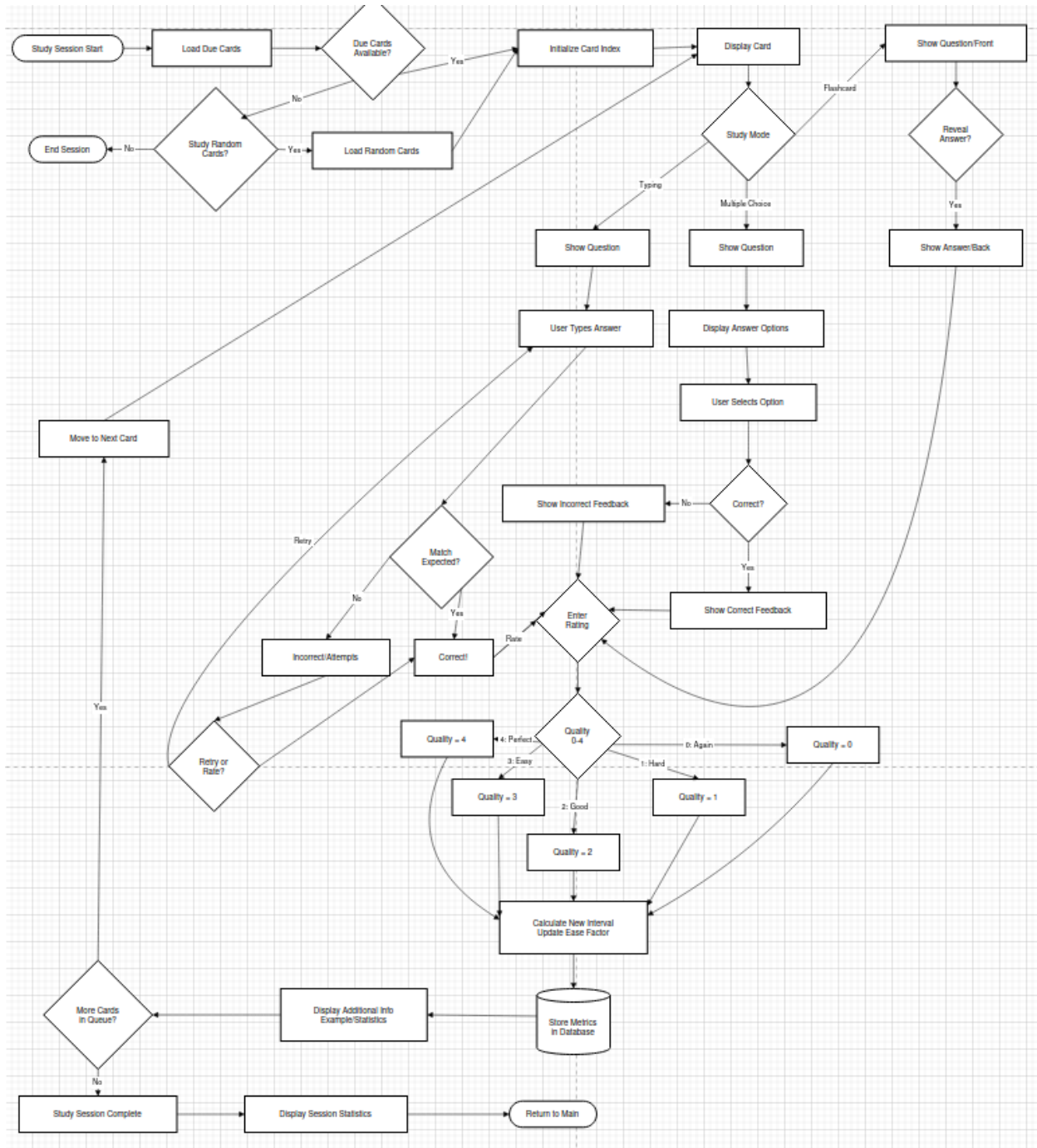


Fig 2.3.2: Adding a word to a list

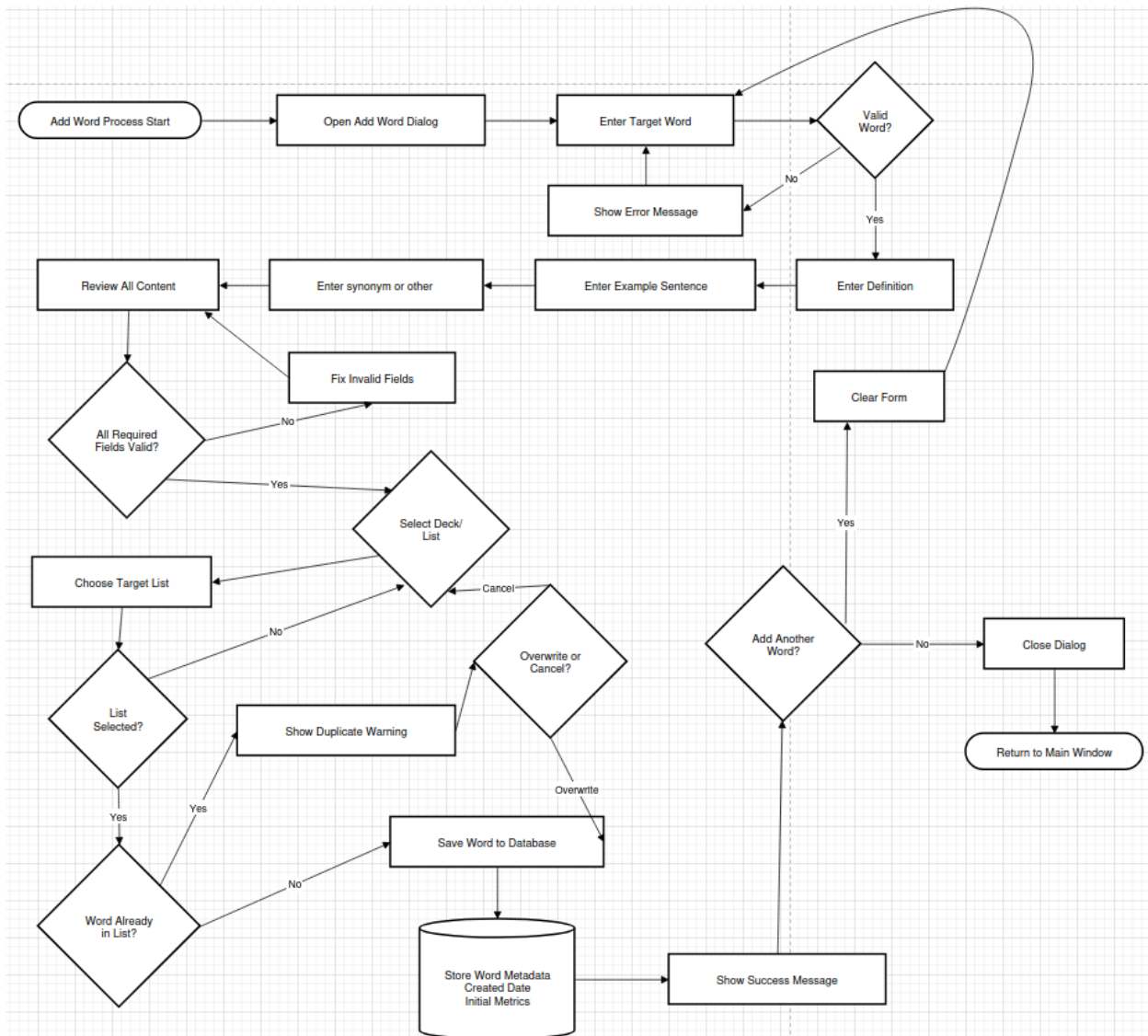
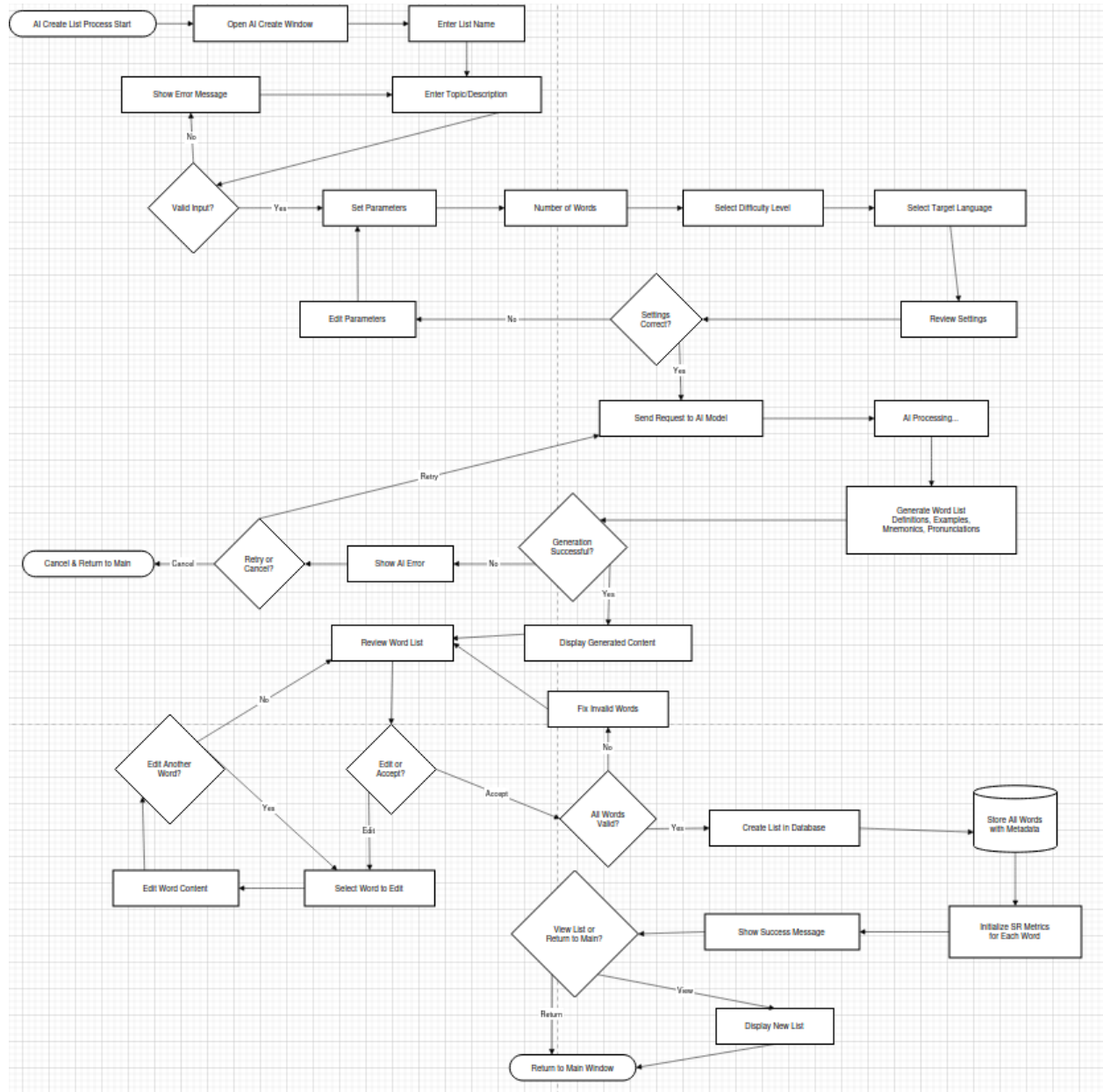
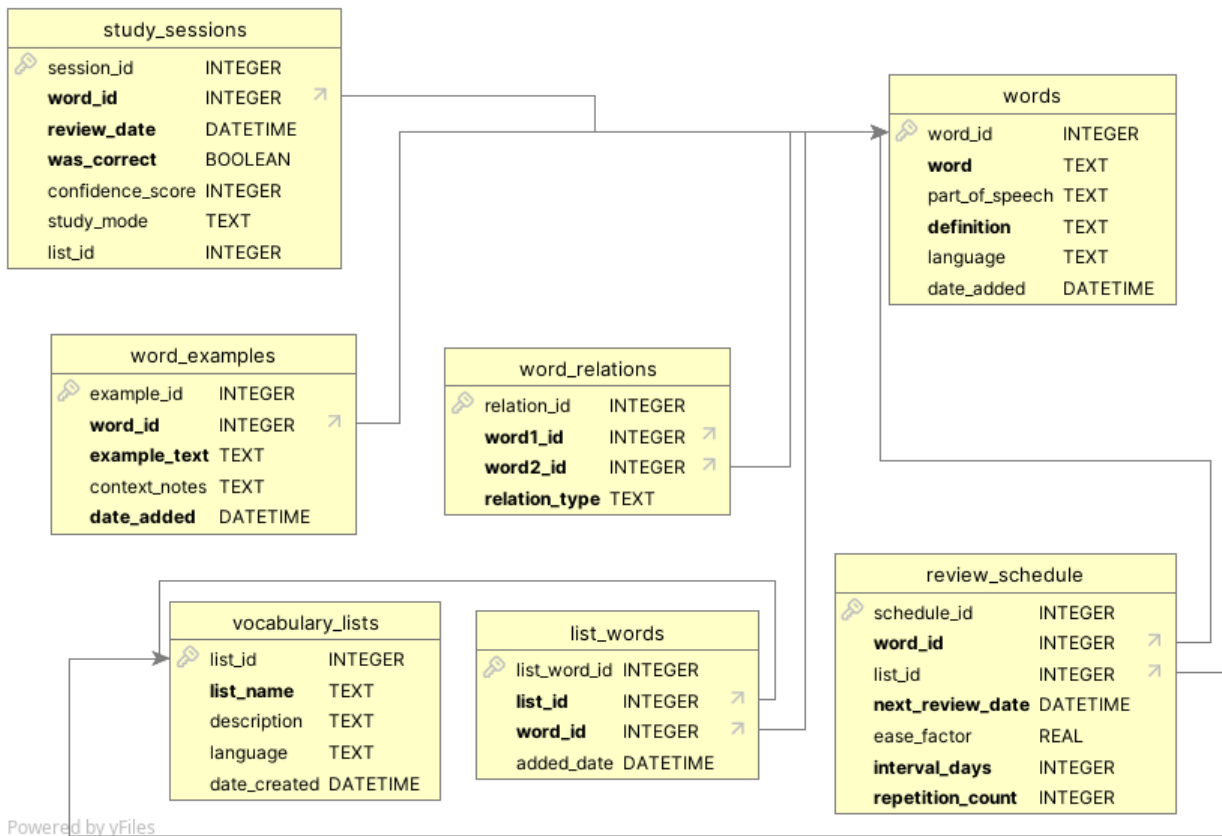


Fig 2.3.3: Creating a list through the use of AI



2.4: Data Structures

ERD Diagram



2.5: Database Dictionary

vocabulary_lists

Column Name	Data Type	Description
list_id	Integer	Unique identifier for each vocabulary list
list_name	Text	The name of each vocabulary list
description	Text	The description of the list
language	Text	The language studied
date_created	DATETIME	When the list was created

list_words

Column Name	Data Type	Description
list_word_id	Integer	The id of the connection between a word and a list
list_id	Integer	The id of the list the word is connected
word_id	Integer	The id of the word that is connected to the list
added_date	DATETIME	The date when the word was added

word_examples

Column Name	Data Type	Description
example_id	Integer	The id of the example
word_id	Integer	The id of the word that the example is connected to
example_text	Text	The example itself
context_notes	Text	Any notes on the example
date_added	DATETIME	When the example was created

word_relations

Column Name	Data Type	Description
relation_id	Integer	The id of the relation
word1_id	Integer	The first word in the relation
word2_id	Integer	The word with which the initial word is related
relation_type	Text	The relation, eg. synonym

study_sessions

Column Name	Data Type	Description
-------------	-----------	-------------

session_id	Integer	The id of the study session
word_id	Integer	The id of the word studied
review_date	DATETIME	The date when studied
was_correct	Boolean	Whether the user was correct when studying
response_time	Integer	How long it took the user to answer
confidence_score	Integer	The confidence score for the supermemo algorithm
study_mode	Text	The study mode the user chose
list_id	Integer	The list from which the word came from

words

Column Name	Data Type	Description
word_id	Integer	The id of the word
word	Text	The word itself
part_of_speech	Text	The part of speech the word is
definition	Text	The definition of the word
language	Text	The language the word is

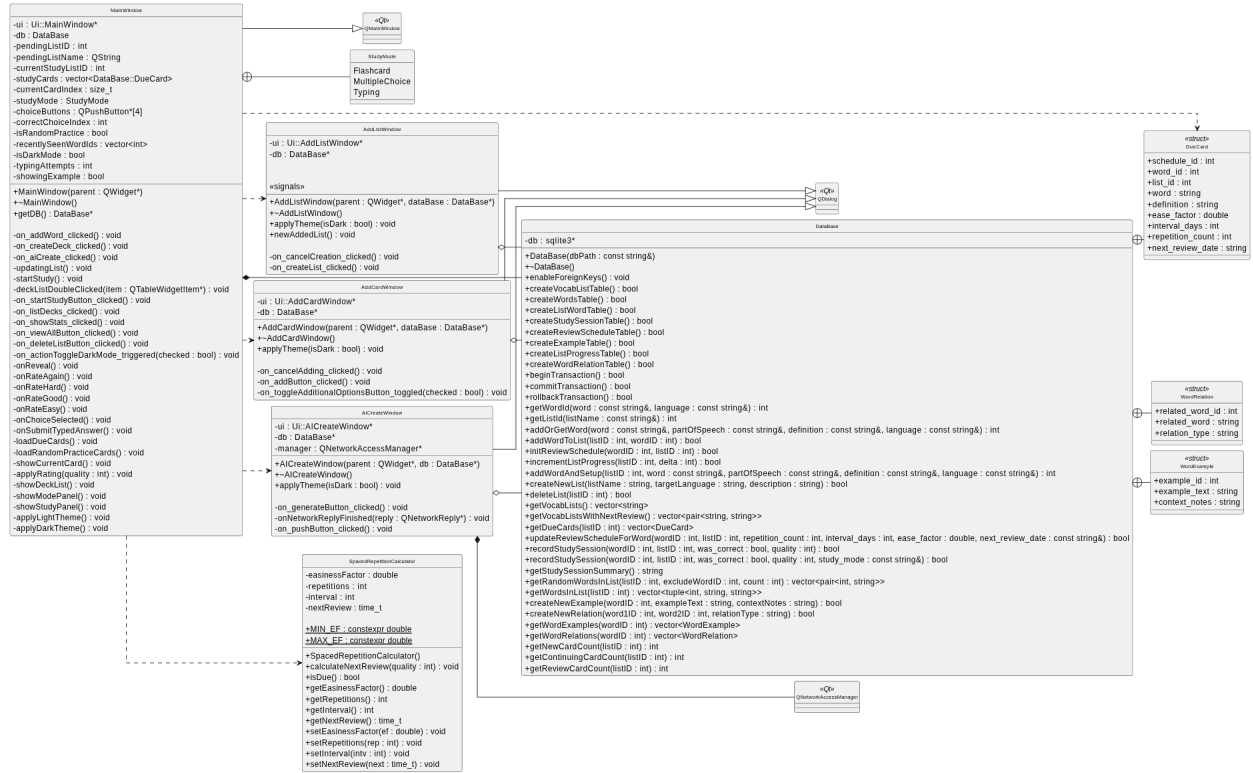
		studied
date_added	DATETIME	The date the word was added

review_schedule

Column Name	Data Type	Description
schedule_id	Integer	The id of the next scheduled time for review
word_id	Integer	The word to be studied
list_id	Integer	The list from which the word comes
next_review_date	DATETIME	The next date for review
ease_factor	Real	The ease factor based on user response, used to calculate next review date
interval_days	Integer	The number of days in between studies
repetition_count	Integer	The number of times a word has been repeated

2.6: Algorithms:

UML Diagram



2.7: Test Plan

Action to be tested	Test method	Expected Result	Success Criteria
User can add more than 1000 vocabulary words to a list, and study it without any issues	Using the import feature within the application, add a vocab list that contains more than 1000 entries, and try to study the list and add words	The user will be able to study and add words to the list without any issues.	1
User can study at least 3 world languages	Test whether typing in several different alphabets/writing systems is possible	The user should have no issues with accents or other typing systems	2
User can study in at least 3 testing modes	Test whether the user can use flashcards, multiple choice, and typing study without any errors	The user should have no issues using any of the three methods of studying	3
User can study from english to a foreign language and from a foreign language to english	Test inputting foreign languages and english within a list and whether it will return the word and its definition properly	The should have no issues using bidirectional study	4
The application should have a spaced repetition	After studying a list, check whether the user is told how many words	The user will have a number of words that are new to study, and words	5

system that tells the user when to study	there are left to study.	that need to still be studied	
The application should have multiple ways of storing a word apart from just the ///definition	Test whether the user can add a word with its definition, synonyms or other connections, and even examples	The user should have no issue with any connections being added and they will be displayed when studying and getting something incorrect	6
The user can view a summary of their studying over some period of time	Test whether the stats buttons works when clicked and it displays data correctly	The user should have no problem viewing statistics	7
The user can generate a list using the integrated AI model within the application	Test whether the user can use pick the AI model and ask it generate a list of a certain amount of words	The user should have no issues generating a list using AI	8
The user must be able to create a list and study it within 10 clicks of opening the application to ensure simplicity	Test how many clicks it takes to add a list and study it after opening the application	The user should need not more than 10 clicks to create a list	9
The user can access lists after closing the application	Test whether all data persists after closing the application	The user will be able to close the application and reopen it to see all of their vocabulary lists and words	10

